

Low-Latency Systems & High-Frequency Trading Foundations

10-Week Step-by-Step Curriculum: Starting with Zero Knowledge to Building an Institutional Limit Order Book

Target Mastery: Market Mechanics, Computer Architecture, Lock-Free Concurrency, Microstructure

Student / Candidate: Akash

Time Commitment: 1.5 to 2.0 Hours / Day (Self-Paced)

Target Outcome: Goldman Sachs Core Strats / HFT Superday Ready

The Ground-Zero Mindset: Starting with zero prior knowledge is your greatest competitive advantage. You do not have bad habits or false assumptions to unlearn. Many university students learn high-level abstractions where memory is 'free' and threads are 'automatic'. In this curriculum, you will learn **Mechanical Sympathy**—how CPUs, hardware caches, memory controllers, and operating system schedulers actually function—and apply it directly to building an electronic exchange from scratch.

1. The 10-Week Curriculum Roadmap at a Glance

Week	Core Concept / Theory	Hands-On Engineering Deliverable	Milestone Code
Week 1	Market Mechanics: Bids, Asks, Spread, FIFO Price-Time Priority.	Textbook Order Book with clean OOP & 5 unit tests.	Phase 1 Baseline
Week 2	Computer Architecture: L1/L2/L3 Caches, Cache Lines, Pointer Chasing.	Array vs. Linked List benchmark; fixed-point integer ticks.	Mechanical Sympathy
Week 3	Zero-Allocation Memory: Slab Allocator, Object Pools, Flat Arrays.	Slab allocator & direct-indexed price array (0 heap malloc).	Phase 2 Zero-Alloc
Week 4	Benchmarking Rigor: Coordinated Omission, RDTSC, HdrHistogram.	Asynchronous Poisson load generator & PMU hardware profiling.	Latency Percentiles
Week 5	Lock-Free Concurrency: Single-Writer Principle, SPSC Queue, False Sharing.	Lock-free SPSC Ring Buffer with alignas(64) & core pinning.	Phase 3 Disruptor
Week 6	Binary Exchange Protocols: NASDAQ OUCH 5.0 & ITCH 5.0 Multicast.	Zero-copy binary packet parser & UDP market data disseminator.	Wall Street Codecs
Week 7	Exchange Mechanics: Call Auction (Walrasian Uncrossing) & Pro-Rata.	Equilibrium clearing price engine & CME-style pro-rata rules.	Call Auction FSM
Week 8	Microstructure & Quant: Stoikov Micro-Price, VPIN Toxicity, MM Bot.	Autonomous Avellaneda-Stoikov market maker & VPIN calculator.	Quant Simulation
Week 9	System Hardening: Deterministic Event Sourcing & Memory-Mapped Log.	Append-only mmap journal with crash-recovery replay test.	L3 Web Visualizer
Week 10	Interview Mastery: Superday Defense, Tradeoff Explanations, Portfolio.	GitHub documentation, benchmark comparison report & mock defense.	Superday Ready

2. Weeks 1 - 3: Foundations, Mechanical Sympathy & Zero-Allocation

Week 1: Financial Market Mechanics & The Baseline Order Book

- **Concepts:** Double auction marketplaces; Bid vs. Ask; The Spread; FIFO Price-Time Priority; Market vs. Limit vs. Cancel orders.
- **Study Resources:** Watch *'How Does a Limit Order Book Work?'* (CME Group, YouTube); read Larry Harris's *Trading and Exchanges* (Chapters 1-3).
- **Hands-On:** Implement Phase 1 in C++ (`std::map<uint64_t, std::list<Order>>`) or Java (`TreeMap`). Write 5 unit tests for exact fills, partial fills, multi-level sweeps, cancellations, and edge cases. Save in `src/baseline/`.

Week 2: Computer Architecture & The 'Mechanical Sympathy' Epiphany

- **Concepts:** The Memory Wall (CPU 0.25ns vs. RAM 100ns); L1/L2/L3 cache hierarchies; 64-byte cache lines; pointer chasing; heap allocation latency.
- **Study Resources:** Watch Carl Cook's landmark talk *'When a Microsecond Is an Eternity'* (CppCon 2017); read CS:APP (Chapter 6: Memory Hierarchy).
- **Hands-On:** Write a standalone benchmark comparing sequential array traversal vs. linked list traversal (see the 10x cache speedup!). Convert order prices from floating-point to 64-bit fixed-point integer ticks.

Week 3: Zero-Allocation Memory Design & Flat Data Structures

- **Concepts:** Object pools; contiguous slab allocators; free-list index stacks; direct array indexing for BBO spread; intrusive doubly-linked lists.
- **Study Resources:** Read Niklas Frykholm's *'Memory Pools and Slab Allocation'*; review boost intrusive list architectures.
- **Hands-On:** Implement `SlabAllocator.hpp` (pre-allocates 1M orders) and `DirectPriceLadder.hpp` (flat array indexed by price offset). Wire into `OrderBookOptimized.hpp` with zero malloc calls on the critical matching path.

Week 1-3 Concept Check: Can You Answer These?

1. Why does reading memory from RAM take hundreds of CPU clock cycles while L1 cache takes only 4 cycles?
2. Why does a linked list suffer more cache misses than a contiguous array of the same size?
3. How does pre-allocating an object pool at startup eliminate latency spikes during trading hours?

3. Weeks 4 - 6: Latency Benchmarking, Lock-Free Concurrency & Protocols

Week 4: Nanosecond Benchmarking & Latency Tail Analysis

- **Concepts:** Average latency fallacy; why p99/p99.99 tails matter; Gil Tene's Coordinated Omission; CPU cycle counters (RDTSC); PMU hardware counters.
- **Study Resources:** Watch Gil Tene's '*How NOT to Measure Latency*' (essential viewing); read Brendan Gregg's Linux perf profiling guides.
- **Hands-On:** Build an asynchronous benchmark driver scheduling Poisson order arrivals; record latencies via HdrHistogram. Run perf stat to measure the 8x drop in L1-dcache misses between Phase 1 and Phase 2.

Week 5: Lock-Free Concurrency & The Single-Writer Principle

- **Concepts:** Mutex lock contention; kernel sleep/wake costs; MESI cache coherence bus storms; Single-Writer Principle; memory order barriers (acquire/release); False Sharing.
- **Study Resources:** Read the *LMAX Disruptor Technical Whitepaper*; watch Herb Sutter's '*Lock-Free Programming*' (CppCon 2014).
- **Hands-On:** Implement a lock-free Single-Producer Single-Consumer (SPSC) ring buffer with alignas(64) cache-line padding. Pin the matching thread to Core 2 via pthread_setaffinity_np. Scale throughput past 5M orders/sec.

Week 6: Binary Exchange Protocols (NASDAQ OUCH & ITCH)

- **Concepts:** Why Wall Street avoids JSON/REST; binary serialization; packed byte structs; network big-endian vs. little-endian; zero-copy deserialization.
- **Study Resources:** Read the official NASDAQ specifications: '*NASDAQ ITCH 5.0*' and '*NASDAQ OUCH 5.0*'.
- **Hands-On:** Implement OuchCodec.hpp to parse binary order entry packets without string allocations; build ItchPublisher.hpp to broadcast UDP multicast Level 3 market data events.

Week 4-6 Concept Check: Can You Answer These?

1. What is Coordinated Omission, and how does a synchronous load tester hide server stalls?
2. What happens to CPU performance when two threads write to variables in the same 64-byte cache line?
3. Why can casting a raw byte pointer to a packed C++ struct be 100x faster than parsing JSON?

4. Weeks 7 - 10: Exchange Mechanics, Microstructure & Superday Mastery

Week 7: Advanced Exchange Mechanics (Call Auctions & Pro-Rata)

- **Concepts:** Continuous trading vs. pre-open Call Auction; Walrasian equilibrium clearing price discovery; CME Pro-Rata allocation; Iceberg orders; Self-Trade Prevention.
- **Study Resources:** Read CME Group's 'Pro-Rata Allocation Rules in Futures'; review NASDAQ Rule 4752.
- **Hands-On:** Build CallAuction.hpp computing the equilibrium price maximizing executed volume; implement configurable FIFO vs. Pro-Rata matching rules.

Week 8: Market Microstructure & Algorithmic Trading Sandbox

- **Concepts:** Sasha Stoikov's Micro-Price; multi-level queue depth imbalance; VPIN order flow toxicity; Avellaneda-Stoikov market making model; inventory risk.
- **Study Resources:** Read Sasha Stoikov's 2018 paper 'The Micro-Price'; read Avellaneda & Stoikov (2008) 'High-frequency trading in a limit order book'.
- **Hands-On:** Implement MicroPrice.hpp and VpinCalculator.hpp. Build an autonomous market maker bot trading against synthetic Poisson noise orders.

Week 9: System Hardening, Real-Time Visualizer & Determinism

- **Concepts:** Deterministic Event Sourcing; memory-mapped files (mmap); crash recovery; real-time web streaming via WebSocket/WebTransport.
- **Hands-On:** Implement append-only mmap binary transaction journaling. Run a 10M trade crash-recovery test asserting bit-for-bit state checksum parity. Connect a React/Canvas Level 3 visualizer.

Week 10: Superday Interview Defense & Portfolio Packaging

- **Deliverables:** Format README with architecture diagrams and Phase 1 vs. 2 vs. 3 benchmark tables; practice the 2-minute project defense; conduct mock technical interviews.
- **Target Result:** Stand out as a top 1% candidate for Goldman Sachs Core Strats / Electronic Trading and premier prop trading desks.

5. Recommended Daily Routine (1.5 - 2.0 Hours / Day)

Time Allocation	Focus Activity	Actionable Goal
First 30 Mins	Concept & Reading	Watch 1 video or read 1 paper/chapter; annotate key concepts.
Next 60 Mins	Hands-On Coding	Write modular, test-driven code for that week's deliverable.
Final 20 Mins	Profiling & Reflection	Run benchmarks, record metrics, and write a 3-bullet takeaway.